TER L1 -> L0 Radix-Refined Root Architecture

v2.1-CODEX - Ultra-Refined Normative Engineering Specification

Subset Sum TER project

24 August 2026

Contents

1 Authority, revision status, and zero-ambiguity rule **2**
1.1 Normative keywords . . . 3

2 Performance envelope before architecture **3**
2.1 Pair-work reduction envelope . . . 3
2.2 Root speed sensitivity . . . 4

3 Scope and non-goals **4**

4 Exact root mathematics **4**

5 Canonical production funnel **5**

6 Global fail-closed contract **6**
6.1 Repository-status reconciliation . . . 6

7 Data model: hot summaries, cold exact state **6**

8 Stage A - RootAux construction **7**

9 Stage B - exact low-k projection **7**

10 Stage C - radix/CSR partition substrate **7**
10.1 Frozen P2 interface . . . 7
10.2 Exact postcondition . . . 8
10.3 Backend rollout frozen by v2.1 . . . 8
10.4 Source reuse binding . . . 9
10.5 Physical digit rules . . . 9
10.6 Partition memory math . . . 9
10.7 Partition speed math . . . 10

11 Stage D - exact region/bucket pruning **10**

12 Stage E - carry-aware exact refinement **11**

13 Stage F - surviving group descriptor **11**

14 Stage G - concrete witness recovery **11**

15 Stage H - optional Blocked Bloom Filter **11**

16 Stage I - exact C candidate directory **12**

17 Stage J - exact low64 gate **12**

18 Stage K - optional full-width additive fingerprint **12**

19 Stage L - arbitrary-width original-residue proof **12**

20 Stage M - canonical TER structural validation **12**

21 End-to-end complexity contract **12**

22 Repository integration map **13**

23 Parallel execution **13**

24 Planner policy **14**

25 Telemetry contract **14**

26 Differential testing matrix **14**

27 Benchmark and falsification protocol **15**

28 Implementation phases and hard gates **15**
28.1 P0 - preserve and replay . . . 15
28.2 P1 - RootAux and arbitrary-width exact tail . . . 15
28.3 P2 - partition/CSR substrate . . . 15
28.4 P3 - exact region pruning . . . 16
28.5 P4 - carry refinement . . . 16
28.6 P5 - adaptive witness recovery . . . 16
28.7 P6 - optional BBF . . . 16
28.8 P7 - end-to-end production root . . . 16
28.9 P8 - parallel refined groups . . . 16
28.10P9 - analytical planner . . . 16

29 Hard acceptance gates **16**

30 Rejected shortcuts **16**

31 Canonical decision record **17**

32 Codex pre-implementation acceptance checklist **17**

1 Authority, revision status, and zero-ambiguity rule

Normative status: TARGET / CONVERGED / CODEX-READY. This document is the master architecture contract for the Full TER L1 -> L0 root. It supersedes the supplied v2.0 document only where this v2.1 edition explicitly narrows an implementation choice, resolves a repository mismatch, or binds Stage C to the companion partition specification.

Companion specification: `TER_Hybrid_Partition_Backend_v2.0_CODEX.pdf` is normative for the implementation of Stage C/P2 only. If it conflicts with this document outside Stage C/P2, this document wins. `implementation_plan.md` is the executable task plan and may not weaken either specification.

Repository baseline: committed source is `Namanpm/subset-sum-ter-fft` at commit `3eac235f871c2e4cf07a73152644e6113eaf` (22 Aug 2026). Local uncommitted work is not represented by that commit. The local `ssp_oracle.cpp` copied into the TER worktree is a donor/reference file for radix extraction and MUST NOT be lost, reset, cleaned, overwritten, or assumed to exist on GitHub.

External algorithm reference: IPS2Ra upstream reference is `ips4o/ips2ra`, commit `ef3407632348f58a6ab75832a749cde99f3` as of this revision. The local clone at `C:\Users\naman\Desktop\ips2ra` is read-only reference material. TER MUST NOT become dependent on that repository, TBB, its scheduler, or its build system merely to obtain blockwise redistribution.

One rule for Codex: if source, this specification, the companion partition specification, and the implementation plan disagree, Codex MUST stop at the first contradiction and report it. It MUST NOT silently choose an interpretation.

1.1 Normative keywords

Keyword	Meaning
MUST / MUST NOT	Hard correctness, architecture, or safety requirement.
SHOULD / SHOULD NOT	Strong engineering default. Change only with measured or correctness evidence.
MAY	Optional implementation choice that does not alter exactness.
EXACT	No false-negative rejection under the stated preconditions.
MAYBE	Candidate may continue; never authority to emit L0.
MEASURED	Directly supported by existing benchmark/repository evidence.
MODELLED	Algebraic sensitivity or memory model, not a measured runtime claim.
EXPERIMENTAL	Comparator or candidate backend that is not production-canonical.
DEFERRED	Intentionally excluded from the current implementation gate.

2 Performance envelope before architecture

The n=72 root is the concrete reference geometry:

- $A = 19,109$
- $B = 32,496$
- $C = 33,505$
- $P_0 = |A||B| = 620,966,064$ logical pair probes
- measured low-24 compatible triples: $1,238,960$
- measured root time: approximately 527.7 ms
- measured full-attempt time: approximately 644.1 ms
- measured root share: approximately 81.93%
- implied pair-probe rate: approximately 1.1767×10^9 probes/s

The expected low-24 compatible triple count under a uniform residue model is approximately $1,240,102.53$; the measured count differs by only about -0.0921%. Therefore low-24 compatibility is not a rare accident and a support-only occupancy stage cannot be assumed to prune strongly.

2.1 Pair-work reduction envelope

Let E be the pair-equivalent work actually opened/recovered after exact region/refinement work. Define:

$$\rho_{open} = \frac{E}{P_0}, \qquad S_{pair} = \frac{P_0}{E}.$$

Opened/recovered E	ρ_{open}	Pair-work reduction P_0/E
620,966,064	1.000000	1.00x
100,000,000	0.161039	6.21x
50,000,000	0.080520	12.42x
25,000,000	0.040260	24.84x
10,000,000	0.016104	62.10x
5,000,000	0.008052	124.19x

Opened/recovered E	ρ_{open}	Pair-work reduction P_0/E
2,000,000	0.003221	310.48x
1,238,960	0.001995	501.20x

These are **work ratios, not wall-clock speedups**.

2.2 Root speed sensitivity

For fixed/setup overhead α expressed as a fraction of the old root pair-scan time, local per-open-pair speed factor g, and rare verification fraction δ_{verify}:

$$S_{root} \approx \frac{1}{\alpha + \rho_{open}/g + \delta_{verify}}.$$

For canonical claims, use $g = 1$ until a root-specific benchmark proves otherwise. The separate 15.90x L2 -> L1 tiled crossover result is evidence that locality matters, not a transferable root multiplier.

With $g = 1$ and ignoring the microscopic verification tail:

E	$\alpha = 2\%$	$\alpha = 0.5\%$
100M	5.52x	6.02x
50M	9.95x	11.69x
25M	16.59x	22.09x
10M	27.70x	47.38x
5M	35.65x	76.62x
2M	43.06x	121.64x
1.239M	45.46x	142.95x

The current whole-attempt Amdahl ceiling from making the root cost zero is only about **5.53x** for the measured n=72 attempt. Root-stage speedup and whole-attempt speedup MUST be reported separately.

3 Scope and non-goals

This specification applies only to the **Full TER L1 -> L0 root join**. Lower merges may continue using their existing tiled/reporting machinery.

The production root MUST NOT:

1. materialize $O(|A||B|)$ pair records or pair sums;
2. use a width-triggered fallback that materializes a quadratic pair index;
3. store a second persistent arbitrary-width quotient-limb vector per L1 entry;
4. treat radix partitioning alone as evidence of subquadratic pair-space reduction;
5. treat SS-style pair streams over the entire global pair universe as a scaling solution;
6. treat a Blocked Bloom Filter as a pair-generation reducer;
7. use probabilistic metadata to accept a root;
8. hard-code low24 as a universal projection width;
9. place NTT/convolution on the mandatory production path without new end-to-end evidence;
10. introduce an ML router before multiple exact backends have stable measured trade-offs.

4 Exact root mathematics

Let n be the instance width and r_1 the number of low bits already fixed at L1. Define:

$$Q = n - r_1.$$

For $g \in \{A, B, C\}$, with original residue r_g and child target t_g:

$$q_g = \frac{(r_g - t_g) \bmod 2^n}{2^{r_1}}.$$

The root condition is:

$$q_A + q_B + q_C \equiv \tau \pmod{2^Q}.$$

For an A, B pair, the required C complement is:

$$d_C = \tau - q_A - q_B \pmod{2^Q}.$$

The search engine MAY use exact projections of q such as $q \bmod 2^k$, `low64(q)`, or $q \bmod p$. Final authority returns to the original residue words:

$$r_A + r_B + r_C \equiv T \pmod{2^n}.$$

At n=72, $Q = 47$. At the projected n=108 geometry, $Q \approx 69$, which already exceeds a `uint64_t`-bound quotient architecture.

5 Canonical production funnel

```
L1 A / B / C
   |
   v
RootAux hot metadata: low_k, low64, fingerprint, EntryId
   |
   v
Exact low-k projection
   |
   v
Stage C: radix/CSR partitioning + cache-local layout
   |
   v
Stage D: exact region/bucket rejection
   |
   v
Stage E: carry-aware exact refinement
   |
   v
Small surviving groups
   |
   v
Adaptive exact witness recovery: direct first, SS-style only when selective
   |
   v
Optional BBF NO/MAYBE gate
   |
   v
Exact C candidate directory
   |
   v
Exact low64 gate (unless already subsumed)
   |
   v
```

```
Optional full-width q mod p rejection
   |
   v
Original-residue word-stream root equality
   |
   v
Canonical TER structural validation
   |
   v
L0 verified root / witness
```

NTT/convolution is comparator/research-oracle only.

6 Global fail-closed contract

Only original-residue arithmetic plus the canonical TER structural validator may authorize L0.

- Exact region summaries may reject only with a proved no-false-negative implication.
- BBF negative may reject only after successful complete construction; positive is MAYBE.
- low64 mismatch is exact rejection; match is not acceptance.
- fingerprint mismatch is exact rejection under the documented wrap set; match is MAYBE.
- incomplete direct/SS enumeration never proves emptiness.
- cancellation and resource refusal never become a negative proof.
- every width path preserves exactness without quadratic-memory fallback.

6.1 Repository-status reconciliation

The committed `ter::Status` currently contains `Completed`, `Found`, `NoWitnessThisAttempt`, `AttemptLimitReached`, `InvalidConfiguration`, `InvalidShape`, `GenerationIncomplete`, `ResourceLimit`, `Cancelled`, and `InvariantViolation`.

This v2.1 specification requires two **append-only** additions before the new root subsystem uses them:

```
NotApplicable,
UnsupportedWidth,
```

Existing enum numeric values MUST remain unchanged. `CompletedNoWitness` is **not** added globally. A fully exhausted diagnostic root report uses `Status::Completed` with zero valid roots; the solve attempt retains the existing `NoWitnessThisAttempt`/`AttemptLimitReached` semantics.

7 Data model: hot summaries, cold exact state

Canonical cold state remains the current TER entry/residue representation. No second persistent wide quotient database is allowed.

```
struct RootAuxSoA {
  std::vector<std::uint32_t> low_k;
  std::vector<std::uint64_t> low64;
  std::vector<std::uint64_t> fingerprint;
  std::vector<EntryId> entry_id;
};
```

The root partition subsystem MAY build an ephemeral compact movement view:

```
struct TerPartitionEntry {
  QuotientResidue low_k;  // uint32_t exact projected bits
  EntryId entry_id;       // uint32_t canonical back-reference
};

static_assert(sizeof(TerPartitionEntry) == 8);
static_assert(std::is_trivially_copyable_v<TerPartitionEntry>);
```

This 8-byte AoS is a **movement record**, not a replacement for the canonical RootAux SoA. The cold full TER entry, packed ternary digits, arbitrary-width residues, and provenance do not travel through the partition hot loop.

8 Stage A - RootAux construction

Status: REQUIRED CORE.

Inputs: canonical L1 entries, child targets, root target, n, r_1. Outputs: EntryId-indexed low projection, exact low64, optional additive full-width fingerprint, and the source handles needed by exact verification.

RootAux construction MUST:

1. derive logical quotient fields exactly from original residue words/targets;
2. avoid persistent full-width quotient limbs;
3. preserve one canonical EntryId per L1 entry;
4. use overflow-checked allocation arithmetic from `ter::limits::checked`;
5. return `ResourceLimit`, `Cancelled`, or `InvariantViolation` rather than silently truncating;
6. record preparation bytes/time and source-word counts.

9 Stage B - exact low-k projection

For each entry:

$$q^{(k)} = q \bmod 2^k.$$

The projection is exact, not a hash. A projected mismatch is a safe rejection; a projected match only establishes compatibility.

`k` is a workload parameter. n=72 low24 is a diagnostic reference, not a universal default. The planner is allowed to use a smaller physical first digit and refine only survivors.

10 Stage C - radix/CSR partition substrate

Status: REQUIRED CURRENT REUSE. **Companion authority:** `TER_Hybrid_Partition_Backend_v2.0_CODEX.pdf`.

Stage C groups exact projected keys into contiguous spans and returns stable EntryIds plus CSR-style bucket offsets. It does **not** prove a pair region empty by itself.

10.1 Frozen P2 interface

```
enum class PartitionBackend : std::uint8_t {
  CurrentScatter,
  BlockInPlace,
  Region2Path,  // reserved/deferred comparator
  Adaptive,     // reserved/deferred until benchmarked
};

struct PartitionSpec {
  std::uint8_t shift = 0;
  std::uint8_t bits = 8;
  std::uint32_t threads = 1;
  PartitionBackend backend = PartitionBackend::CurrentScatter;
  std::uint64_t scratch_cap_bytes = 0;
  std::uint32_t block_bytes = 2048;
};

struct PartitionStats {
  PartitionBackend requested = PartitionBackend::CurrentScatter;
```

```
  PartitionBackend effective = PartitionBackend::CurrentScatter;
  std::uint64_t histogram_ns = 0;
  std::uint64_t prefix_ns = 0;
  std::uint64_t scatter_ns = 0;
  std::uint64_t local_classification_ns = 0;
  std::uint64_t block_permute_ns = 0;
  std::uint64_t cleanup_ns = 0;
  std::uint64_t records_classified = 0;
  std::uint64_t records_written = 0;
  std::uint64_t blocks_flushed = 0;
  std::uint64_t blocks_permuted = 0;
  std::uint64_t partial_records_repaired = 0;
  std::uint64_t histogram_bytes = 0;
  std::uint64_t offset_bytes = 0;
  std::uint64_t record_scratch_bytes = 0;
  std::uint64_t block_buffer_bytes = 0;
  std::uint64_t scratch_peak_bytes = 0;
  std::uint64_t logical_bytes_read = 0;
  std::uint64_t logical_bytes_written = 0;
};

Status partition_chunk(
    std::span<TerPartitionEntry> entries,
    const PartitionSpec& spec,
    PartitionWorkspace& workspace,
    std::span<std::uint64_t> bucket_offsets,
    PartitionStats& stats);
```

`bucket_offsets.size()` MUST equal $(2^{bits} + 1)$.

10.2 Exact postcondition

For $B = 2^b$ and

$$bucket(e) = ((e.low_k \gg shift) \ \& \ (B - 1)),$$

successful completion requires:

1. `bucket_offsets[0] == 0`;
2. `bucket_offsets[B] == entries.size()`;
3. offsets are monotone;
4. each input record appears exactly once in output;
5. every input EntryId appears exactly once if input IDs are unique;
6. every record in `[offsets[j], offsets[j+1])` has digit `j`;
7. no stability guarantee inside a bucket;
8. no full sort beyond the requested physical digit;
9. cancellation/resource failure leaves no false proof and returns a non-completion status.

10.3 Backend rollout frozen by v2.1

Current implementation order is not negotiable without benchmark evidence:

1. `CurrentScatter` - first correctness/performance baseline, extracted from the mature radix histogram/prefix/scatter donor.
2. `BlockInPlace` - second implementation, adapted from IPS2Ra blockwise local classification, block permutation, and margin cleanup.
3. `Region2Path` - DEFERRED scientific comparator. It is not required for the first production candidate.
4. `Adaptive` - DEFERRED until paired benchmark data exists. Before then, selecting it returns `NotApplicable`.

`Region2Path` MUST NOT be implemented before `CurrentScatter` and `BlockInPlace` have differential correctness and isolated benchmark data unless the implementation plan is explicitly revised.

10.4 Source reuse binding

The local donor `ssp_oracle.cpp` provides the preferred starting machinery for:

- serial generic histogram/prefix/scatter;
- parallel radix16 private per-worker histograms;
- bucket-major/worker-major prefix construction;
- disjoint destination offsets;
- `alignas(64)` worker-state layout;
- persistent `std::thread` worker pool;
- count/prefix/scatter timing and histogram/offset/scratch telemetry.

The donor sorts **values**, so its kernel MUST be generalized to classify by `entry.low_k` while moving the complete `TerPartitionEntry` record.

IPS2Ra reference components allowed to influence/carry source into the TER BlockInPlace backend are limited to the one-level ideas present in:

- `include/ips2ra/partitioning.hpp`
- `include/ips2ra/local_classification.hpp`
- `include/ips2ra/block_permutation.hpp`
- `include/ips2ra/cleanup_margins.hpp`
- `include/ips2ra/bucket_pointers.hpp`
- selected buffer/memory concepts from `include/ips2ra/memory.hpp`, `buffers.hpp`, and `config.hpp`

The first TER port MUST NOT copy/import IPS2Ra recursive sorting, sampling, TBB scheduler, task queues, base-case sorting, dynamic full-sort recursion, or build-system dependencies.

10.5 Physical digit rules

Logical projection width and physical digit width are separate.

- `CurrentScatter`: initial supported physical widths 8, 12, 16.
- `BlockInPlace` v1: physical width **8 only**. This follows the IPS2Ra default and keeps per-bucket block buffers bounded.
- `Region2Path` if later implemented: start at 8; never start at 16 merely because the donor sorter is called radix16.
- `bits == 0`, `bits > 16`, or `shift + bits > 32` is invalid for the 32-bit movement view.

For the n=72 low24 experiment, the first production-shaped schedule to test is:

```
16-bit CurrentScatter
    -> exact pruning
    -> 8-bit BlockInPlace only inside survivors
```

The following remain benchmark schedules, not production constants:

```
12 + 12
8 + 8 + 8
```

Later digits MUST operate only on exact surviving spans when Stage D/E is active.

10.6 Partition memory math

For CurrentScatter with N 8-byte records, t workers, $B = 2^b$, and 8-byte `size_t`:

$$M_{worker} \approx 2tB \cdot 8,$$
$$M_{global} \approx B \cdot 8,$$
$$M_{records} = 8N.$$

At 16 workers:

bits	buckets	worker hist+offset	global array	dense metadata
8	256	0.0625 MiB	0.0020 MiB	0.0645 MiB
12	4,096	1.00 MiB	0.0313 MiB	1.0313 MiB
16	65,536	16.00 MiB	0.50 MiB	16.50 MiB
24	16,777,216	4,096 MiB	128 MiB	4,224 MiB

A dense 24-bit per-worker histogram is forbidden in this design.

10.7 Partition speed math

Let $H = T_{hist} + T_{prefix}$, $S = T_{scatter}$, and R be the complete BlockInPlace redistribution cost after histogram/prefix. Then:

$$T_{current} = H + S,$$
$$T_{block} = H + R,$$
$$Speedup = \frac{H + S}{H + R}.$$

Break-even is exactly $R < S$. If $H/S = 0.5$:

R/S	Modelled partition speedup
0.50	1.50x
0.70	1.25x
0.80	1.15x
0.90	1.07x
1.00	1.00x
1.20	0.88x
1.50	0.75x

These are sensitivity values only.

11 Stage D - exact region/bucket pruning

Status: REQUIRED CORE NEW.

Stage D is the primary mechanism intended to reduce the opened Cartesian mass. It receives exact bucket spans and exact C target/complement summaries and asks whether a region $A_i \times B_j$ can contain any compatible witness under the currently proven exact bits/carry state.

A region may be rejected only when the summary proves zero compatible witnesses. Permitted exact summary families include bounded-universe bitsets/hierarchical bitsets and sparse exact occupancy directories. Min/max interval overlap is not sufficient as the primary near-uniform-root proof.

Required telemetry:

- regions considered;
- exact regions rejected;
- pair mass represented by rejected regions;
- pair mass opened;
- ρ_{region} and ρ_{open};
- summary bytes and build/refinement time.

12 Stage E - carry-aware exact refinement

If a low chunk has width k, write:

$$a = a_0 + 2^k a_1, \quad b = b_0 + 2^k b_1.$$

The carry is:

$$\kappa_0 = \left\lfloor \frac{a_0 + b_0}{2^k} \right\rfloor.$$

The next exact chunk condition is:

$$a_1 + b_1 + \kappa_0 \equiv d_1 \pmod{2^h}.$$

Every refinement node MUST carry `bits_proven`, carry state, A/B spans or compact ID lists, exact target class, conservative pair mass, and summary handle. Child witness sets must exactly partition the parent witness set.

Mandatory boundary tests: 7/8, 15/16, 31/32, 63/64, multi-carry chains, modulus wrap.

13 Stage F - surviving group descriptor

A surviving group is MAYBE work, never an answer.

```
struct RefinedGroup {
  std::uint32_t bits_proven;
  CarryState carry;
  EntrySpan a_ids;
  EntrySpan b_ids;
  TargetClass targets;
  std::uint64_t conservative_pair_mass;
  SummaryHandle exact_summary;
};
```

The implementation MUST report group count, median/max pair mass, total residual Cartesian mass, and bits-proven distribution.

14 Stage G - concrete witness recovery

Direct nested loops are the first backend for tiny groups. SS-style ordered pair streams are permitted only for already-narrowed groups when a measured/modelled cost predicts fewer visited pair-equivalents than direct enumeration.

Global SS over the full A x B universe is forbidden as a production scaling claim.

Recovery outputs concrete `(A_id,B_id,complement_key)` suspects and never authorizes L0.

15 Stage H - optional Blocked Bloom Filter

BBF is optional and benchmark-gated. It is built on the fixed C side only after an exact C lookup structure exists.

- negative: exact rejection only if all required C keys were inserted successfully;
- positive: MAYBE;
- allocation/build failure: bypass or `ResourceLimit`, never "all keys absent";
- delete/bypass BBF if exact C CSR lookup is faster end-to-end.

16 Stage I - exact C candidate directory

The root requires concrete C EntryIds. Preferred representation is a radix/CSR directory keyed by the exact projection already proven.

Required properties:

- deterministic construction;
- contiguous ID spans;
- exact fan-out for duplicate keys;
- no dependence on BBF;
- no full-width hash key on the hot path unless later evidence justifies it.

17 Stage J - exact low64 gate

Cache $q \bmod 2^{\min(Q,64)}$. For a concrete triple, reject on mismatch. If `bits_proven >= min(Q,64)`, skip the redundant gate.

For $Q \leq 64$, low64 may prove the quotient equation, but original-residue verification remains the canonical independent arithmetic authority for the production root.

18 Stage K - optional full-width additive fingerprint

Precompute $F(q) = q \bmod p$ by streaming quotient/source words once. Store one machine scalar per entry.

Because the root equality is modulo 2^Q, a valid integer sum may be $\tau + m2^Q$ for $m \in \{0, 1, 2\}$. Let $R = 2^Q \bmod p$. A candidate passes the fingerprint necessary condition only if:

$$F_A + F_B + F_C \equiv \tau_p + mR \pmod{p}$$

for one allowed m. Collision is MAYBE; mismatch is rejection. The fingerprint is removable if benchmarked value is negligible.

19 Stage L - arbitrary-width original-residue proof

The final arithmetic verifier streams existing little-endian residue words, performs $A + B + C + carry$, masks the top partial word, compares against the root target modulo 2^n, and exits on first mismatch.

It MUST work at widths above 64 without creating a persistent wide quotient array.

20 Stage M - canonical TER structural validation

Arithmetic equality is necessary but not sufficient. Only after exact root arithmetic passes may the implementation load packed ternary state and validate overlap legality, digit counts/distribution, representation invariants, provenance/canonicalization, and output policy.

`FirstWitness` returns only after arithmetic and structural validation both succeed.

21 End-to-end complexity contract

Let $N = a + b + c$, $P_0 = ab$, Φ be exact front-end/refinement work, E actual pair-equivalent recovery work, β exact C lookup suspects, Z concrete triples, S exact-word survivors, V structural survivors, $L_n = \lceil n/W \rceil$ residue words, and D packed TER words.

With direct recovery:

$$T_{root} = O(NL_q + \Phi + E + \beta + Z + SL_n + VD).$$

With selective SS-style recovery:

$$T_{root} = O(NL_q + \Phi + E \log H + \beta + Z + SL_n + VD).$$

The intended regime is $S, V \ll Z \ll E \ll P_0$. If E remains near P_0, the root scaling objective has failed even if locality improves wall time.

Memory MUST remain pairless:

$$M_{root} = O(N) + M_{buckets} + M_{refine} + M_{recovery} + M_{BBF} + \text{existing cold state}.$$

Forbidden memory: full pair arrays, full pair sums, second persistent arbitrary-width quotient limbs, 2^Q direct-address tables at large Q, and unbounded witness queues.

22 Repository integration map

Current component	Existing role	v2.1 target
`include/ter/full_ter.hpp`	Root capture/config/telemetry/output policy	Add root-v2 config handles and stage telemetry without breaking current APIs.
`src/full_ter.cpp`	Current exact tiled M2 root and legacy fallback paths	Integrate the new funnel behind explicit feature/config gates; preserve current exact comparator path during differential rollout.
`include/ter/limitations.hpp`	Central runtime policy registry	Own every new numeric threshold/default; no duplicated constants in algorithms.
`include/ter/types.hpp`	`EntryId`, `Status`, global enums	Append `NotApplicable`, `UnsupportedWidth`; preserve existing values.
`tests/full_ter_tests.cpp`	Root correctness/replay tests	Add captured-root differential tests and width-ladder tests.
`CMakeLists.txt`	C++20 targets and strict warnings	Add focused partition/root-v2 sources, tests, benchmark, `Threads::Threads`; do not add TBB/IPS2Ra dependency.
local `ssp_oracle.cpp`	Mature radix donor, not committed	Extract/adapt only required radix/worker/telemetry logic; do not commit the entire donor file as production architecture.
local `C:\Users\naman\Desktop\ips2ra`	Read-only blockwise reference	Copy/adapt narrowly, preserve BSD notice, do not modify/link the repo.

23 Parallel execution

The initial partition backend uses the donor-style persistent `std::thread` worker pool rather than introducing OpenMP/TBB as a mandatory dependency.

- independent refined groups may run in parallel later;
- mutable overlapping global enumeration is forbidden;
- cancellation is cooperative and exact;
- `FirstWitness` cancellation triggers only after fully exact arithmetic and TER validation;
- bounded queues only;
- GPU work is deferred until compact exact work units exist.

24 Planner policy

The initial planner is analytical and deterministic. Before measured partition data exists, `PartitionBackend::Adaptive` returns `NotApplicable`; it does not guess thresholds.

After P2 benchmarks, the planner may choose based on:

- N;
- physical digit width;
- scratch cap/pressure;
- bucket skew/nonempty count/max occupancy;
- available threads;
- measured CurrentScatter vs BlockInPlace model coefficients;
- downstream exact pruning survivor mass.

ML routing remains out of scope.

25 Telemetry contract

Every run MUST separate **work reduction** from **per-work acceleration**.

Minimum root telemetry categories:

Category	Required fields
Geometry	n, r1, Q, A, B, C, orientation, logical pairs
Preparation	bytes, ns, low-k schedule, source-word count, fingerprint modulus if enabled
Partition	backend requested/effective, physical bits, bucket counts/occupancy, histogram/prefix/redistribution ns, scratch bytes
Region	regions considered/rejected, pair mass rejected/opened, rho_open
Refinement	nodes, carry states, bits proven, pair mass before/after, ns
Recovery	direct/SS groups, pair-equivalents visited, heap work, witnesses emitted, budget hits
C lookup	queries, fanout, concrete triples
Filters	BBF/fingerprint/low64 checks, rejects, bypasses
Exact	exact-root calls, words read, mismatch word distribution, passes
Structure	checks, overlap/distribution rejects, valid roots
Memory	declared/observed peak bytes; pair_records MUST remain zero
Outcome	Found, Completed exact diagnostic result, ResourceLimit, NotApplicable, UnsupportedWidth, Cancelled, InvariantViolation

26 Differential testing matrix

The implementation is not promotable without these families:

1. tiny exhaustive roots versus cubic/reference oracle;
2. captured genuine L1 roots versus current exact root behavior;
3. partition property tests with no EntryId drop/duplication;
4. exact region false-negative tests on exhaustive small universes;
5. carry boundaries 7/8, 15/16, 31/32, 63/64 and chained carry;
6. width ladder Q = 1, 24, 32, 47, 64, 65, 69, 96, 127, 128, 129, 256 plus randomized larger widths;
7. recovery witness-set equality on every completed local group;

8. forced fingerprint collisions followed by exact-word rejection;
9. cancellation/resource paths never interpreted as negative completion;
10. deterministic same-capture/same-config canonical result and stable logical work counters.

27 Benchmark and falsification protocol

Every performance claim MUST include setup and exact tail costs. Genuine captured L1 roots are primary evidence.

Required families:

- genuine captured roots;
- planted witness roots;
- random-target/no-witness roots;
- uniform and skewed synthetic quotients;
- imbalanced A/B/C;
- width ladder above 64.

Falsification A: if ρ_{open} remains near 1 on genuine roots, the exact region/refinement front end failed its structural goal.

Falsification B: if Φ is of the same order as avoided pair work, the architecture moved rather than removed the bottleneck.

Deletion rule: optional BBF, fingerprint, physical schedules, and planner heuristics are removed/bypassed if they do not improve total root wall time or memory objectives.

28 Implementation phases and hard gates

28.1 P0 - preserve and replay

- Preserve all local uncommitted work.
- Freeze genuine root captures using the existing `FullTerRootCapture` seam.
- Record baseline logical pairs, root ns, pair probes, output policy, peak bytes, and outcome.

Gate: replay is deterministic and existing tests pass before architecture changes.

28.2 P1 - RootAux and arbitrary-width exact tail

- append required status values;
- implement RootAux projection extraction without persistent wide q;
- exact low64;
- optional fingerprint preparation;
- original-residue arbitrary-width verifier.

Gate: differential equality at width ladder including Q=69+.

28.3 P2 - partition/CSR substrate

P2 is expanded into mandatory subgates:

- **P2A:** one-digit `CurrentScatter` on 8-byte `(low_k,EntryId)` records;
- **P2B:** exact bucket offsets and property tests;
- **P2C:** isolated partition benchmark/telemetry;
- **P2D:** portable 8-bit `BlockInPlace` from IPS2Ra concepts;
- **P2E:** parallel BlockInPlace through existing/std::thread pool, no TBB dependency;
- **P2F:** paired CurrentScatter vs BlockInPlace benchmark and confidence intervals;
- **P2G:** exact C CSR directory on the same EntryId/bucket representation;
- **P2H:** only after P2F, install measured backend thresholds or leave Adaptive `NotApplicable`.

Speed-default gate: lower 95% CI of paired geometric-mean speedup > 1.05x on representative captured roots with no serious tail regression.

Memory-backend gate: at least 4x lower peak partition scratch while within 10% of CurrentScatter wall time.

28.4 P3 - exact region pruning

Implement one exact summary backend. Measure rejected pair mass before opening entries.

Gate: zero false-negative region prunes and materially reduced pair mass on at least one genuine captured root.

28.5 P4 - carry refinement

Extend exact prefixes chunk by chunk with explicit carry state.

Gate: witness conservation against brute force across boundary/carry tests.

28.6 P5 - adaptive witness recovery

Direct tiny-group recovery first. Add SS-style local streams only after measured selective benefit.

Gate: every completed group equals local brute-force witness set.

28.7 P6 - optional BBF

Only after exact C lookup exists. Remove if not faster.

28.8 P7 - end-to-end production root

Wire partition/refine -> recovery -> C -> low64 -> fingerprint -> exact words -> TER -> output policy.

Gate: zero differential mismatch; no quadratic pair allocation; Q>64 remains bounded-memory exact.

28.9 P8 - parallel refined groups

Bounded worker queues and exact cancellation.

28.10 P9 - analytical planner

Use measured cost models. ML remains deferred.

29 Hard acceptance gates

The architecture is not complete until all applicable gates hold:

- **A:** zero differential correctness mismatches;
- **B:** Q=69 and larger never dispatch to quadratic-memory fallback;
- **C:** genuine-root ρ_{open} materially below 1 for the claimed structural result;
- **D:** total root wall time improves after all setup/refinement costs;
- **E:** `pair_records == 0` and peak memory honors configured limits;
- **F:** low24 is not universal and physical refinement adapts to density;
- **G:** NTT absent from mandatory production critical path;
- **H:** every copied/adapted IPS2Ra source portion preserves BSD-2-Clause notice;
- **I:** TER builds on Windows without depending on the unmodified IPS2Ra build system;
- **J:** CurrentScatter remains available as exact differential/performance control during rollout.

30 Rejected shortcuts

Shortcut	Rejection
Mandatory NTT front end	Count/support cost plus witness recovery; evidence does not justify production dependency.
Flat Bloom + scan all pairs	Still O(

Shortcut	Rejection
Global SS streams	Memory-light but can still visit the entire pair space with heap overhead.
Literal 16-bit Regions graph	65,536 graph countries and cache-hostile metadata; not production default.
Import IPS2Ra wholesale	Windows/TBB/build coupling and recursive full-sort machinery not needed by TER.
Copy entire `ssp_oracle.cpp` into production target	Pulls unrelated oracle complexity into TER and obscures ownership.
Full 69-bit direct-address table	Physically impossible at target scale.
Store full quotient limbs per entry	Duplicates exact state and harms bandwidth.
Hard-code 128-bit exact root	Merely moves the width cliff.
GPU brute force as scaling solution	Faster quadratic work remains quadratic.
Adaptive thresholds before benchmark	Optimizes guesses.

31 Canonical decision record

Decision	v2.1 choice
Root objective	Reduce opened pair mass before pair recovery.
Root partition contract	One physical digit at a time, EntryId-preserving CSR spans.
Initial partition baseline	CurrentScatter adapted from donor radix machinery.
Primary in-place candidate	IPS2Ra-derived BlockInPlace, physical radix8 first.
Regions graph	Deferred scientific comparator, not initial implementation dependency.
Adaptive backend	Deferred until paired measurements exist.
Root metadata	Canonical SoA; optional ephemeral 8-byte partition record.
Higher-bit narrowing	Carry-aware exact refinement.
Witness identity	Direct local recovery; SS-style only selectively.
Bloom	Optional NO/MAYBE gate only.
Wide exact state	Existing original residue words.
Final arithmetic	Word-stream modulo 2^n.
Final legality	Canonical TER structural validator.
Parallel substrate	Existing/std::thread worker machinery; no mandatory TBB/OpenMP.
Production fallback	Fail closed or exact bounded alternate backend only.

One-line contract: radix/partitioning determines **where** exact work is organized; exact region/carry refinement determines **how much** pair mass is opened; direct/SS recovery identifies **who** the witnesses are; low64/fingerprint/BBF only reject cheaply; original residue words plus TER structural validation alone decide the root.

32 Codex pre-implementation acceptance checklist

Before editing code, Codex MUST be able to answer YES to every item:

☐ I have read this entire v2.1 specification.
☐ I have read the entire companion hybrid partition specification.
☐ I have read `implementation_plan.md`.
☐ I have inspected the current local working tree and will not reset uncommitted `ssp_oracle.cpp`.
☐ I know the committed baseline is `3eac235f...` but local files may be newer.
☐ I have inspected `include/ter/types.hpp`, `limitations.hpp`, `full_ter.hpp`, `src/full_ter.cpp`, current tests, and `CMakeLists.txt`.
☐ I have inspected the donor radix functions/worker telemetry in local `ssp_oracle.cpp`.

☐ I have inspected the listed IPS2Ra files read-only and recorded their license obligations.
☐ I understand that the first partition deliverable is CurrentScatter, not BlockInPlace.
☐ I understand that Region2Path and Adaptive are deferred until the specified gates.
☐ I will run differential tests before timing.
☐ I will not claim modelled speedups as measured results.